

Dynamic Programming & Reinforcement Learning

Finite-Horizon DP and Online Planning

Instructor: Chuanhao Li

October 20, 2026

Content for Today

1. Finite-horizon dynamic programming 5–29

A deadline, stage values, and backward computation

2. Computing an action when it is needed 30–62

Computation cost, local search, simulated episodes, and MCTS

3. Where the course goes next 63–69

Representation, model knowledge, and data access

What Lectures 1–4 have established

We have studied a **known, time-homogeneous**, fully observed MDP with finite state and action sets. For a stationary Markov policy π,

$$V^{\pi}(x) = \mathbb{E}_x^{\pi}\left[\sum_{t=1}^{\infty} \gamma^{t-1} R_t\right], \qquad 0 \le \gamma < 1.$$

- Bellman equations describe policy values and optimal values.
- A deterministic stationary Markov policy can be optimal.
- VI and PI compute value tables and action choices across the state space.

Today: what changes when there is a deadline, or when computing the complete solution is too expensive?

A different endpoint, the same decision process

Previously: no fixed final decision

$$\mathbb{E}_x^\pi\left[\sum_{t=1}^{\infty}\gamma^{t-1}R_t\right].$$

Discounting makes bounded rewards summable.

Today: exactly H decision stages

$$\mathbb{E}_x^\pi\left[\sum_{t=1}^{H}R_t + g(X_{H+1})\right].$$

Finite sums need no discount. The optional final payoff g may be zero.

- Values and optimal actions may depend on the stage, even with unchanged dynamics: $V_t^\pi(x)$, $\pi_t(x)$.
- The known terminal boundary starts a backward recursion. We do not need an infinite-iteration limit.

Unchanged: the same known reward and transition rules apply at every stage. Only the number of remaining decisions changes.

Finite-Horizon Dynamic Programming



A deadline gives the problem a boundary

There are H decisions, at stages $t = 1, \ldots, H$.

$$X_1 \xrightarrow{A_1, R_1} X_2 \longrightarrow \cdots \longrightarrow X_H \xrightarrow{A_H, R_H} X_{H+1}$$

optional payoff
$g(X_{H+1})$

At stage t, the remaining return is

$$\boxed{G_t := \sum_{s=t}^{H} R_s + g(X_{H+1}), \qquad G_{H+1} = g(X_{H+1}).}$$

The terminal payoff is paid **once**. There is no decision at $H + 1$.

A terminal payoff is optional

g specifies an additional payoff determined by the final state.

No additional payoff

Keep the stage rewards $R_1, \ldots, R_H$:

$$g(x) = 0.$$

Example: leftover inventory

After the final selling day, each remaining item sells for 2. With x items left,

$$g(x) = 2x.$$

This is added to earlier sales revenue.

Keeping g separate lets the ordinary reward rule stay the same at every stage.

Why was there no separate g before?

Our infinite-horizon discounted objective has no prescribed final time. All rewards belong in the infinite sum. A reward at termination can be included in the corresponding R_t.

Time and history are available to a policy

The state set $\mathcal{X}$ is finite. At state x, the feasible action set $\mathcal{A}(x)$ is finite and nonempty.

Before decision t, the observed history is

$$H_t = (X_1, A_1, R_1, \ldots, X_{t-1}, A_{t-1}, R_{t-1}, X_t).$$

An admissible policy ρ chooses a distribution $\rho_t(\cdot \mid H_t)$ on $\mathcal{A}(X_t)$, using no future information.

A **stage-Markov policy** is a sequence

$$\pi = (\pi_1, \ldots, \pi_H), \qquad \pi_t(\cdot \mid x) \text{ a distribution on } \mathcal{A}(x).$$

A deterministic stage-Markov policy selects one action $\pi_t(x)$. Different remaining times may call for different actions at the same state.

We allow the full history as before. Later we will **prove** that the stage-Markov subclass contains an optimal policy.

The same known model at every stage

Given the current state and action, the **known joint law** of (R_t, X_{t+1}) is the same at every stage and does not depend on the earlier history.

Write $r(x, a)$ for the known mean reward and $\mathcal{P}(x' \mid x, a)$ for the known next-state probabilities. In particular,

$$\mathbb{E}[R_t \mid H_t, A_t] = r(X_t, A_t),$$
$$\Pr(X_{t+1} = x' \mid H_t, A_t) = \mathcal{P}(x' \mid X_t, A_t),$$

almost surely (with probability one). Assume $|R_t| \le R_{\max}$ and $|g(x)| \le g_{\max}$. Reward and next state may be correlated.

No stage index on r, $\mathcal{P}$, or $\mathcal{A}(x)$. The deadline changes how many decisions remain, not the physical rules.

A value for every stage and state

Start a tail problem at stage t in state x. For a stage-Markov policy π, let

$$V_t^\pi(x) := \mathbb{E}_{t,x}^\pi\left[\sum_{s=t}^{H} R_s + g(X_{H+1})\right].$$

$\mathbb{E}_{t,x}^\pi$ initializes $X_t = x$ and follows $\pi_t, \dots, \pi_H$. The optimal tail value allows **all** admissible tail policies:

$$V_t^*(x) := \sup_\rho \mathbb{E}_{t,x}^\rho[G_t], \qquad V_{H+1}^*(x) = g(x).$$

(t, x) leaves $H - t + 1$ decisions. For one state and one action with reward 1 and $g = 0$, $V_t^\pi(x) = H - t + 1$: the value changes even though the model and policy do not.

We will construct candidate tables U_t backward from $U_{H+1} = g$, then prove that $U_t = V_t^*$.

Evaluate a stage-Markov policy backward

Fix a stage-Markov policy $\pi = (\pi_1, \dots, \pi_H)$. The return splits into the immediate reward and the remaining return:

$$G_t = R_t + G_{t+1}, \qquad V_{H+1}^{\pi}(x) = g(x).$$

Condition on the action, then the next state, as in our earlier Bellman calculation:

$$V_t^{\pi}(x) = \sum_{a \in \mathcal{A}(x)} \pi_t(a \mid x) \left[r(x,a) + \sum_{x' \in \mathcal{X}} \mathcal{P}(x' \mid x, a) V_{t+1}^{\pi}(x') \right].$$

- Compute $t = H, \dots, 1$: each step uses the already known **next-stage** table V_{t+1}^{π}.
- For a deterministic policy, the action average selects $a = \pi_t(x)$.
- This state-only recursion applies to stage-Markov π. An arbitrary history policy ρ may require conditioning on H_t.

A job with a deadline: when is work worth doing?

A job consists of **two units of work**. We have two work periods, $t = 1, 2$, and start with no completed units: $X_1 = 0$.

- In each period, we may work or wait. For now, working completes one unit with certainty and costs 1.
- A completed job earns a bonus of 10 at the deadline, after period 2. An unfinished job earns no bonus.

We choose when to work to maximize the total payoff:

$$G_1 = R_1 + R_2 + g(X_3) = \text{completion bonus} - \text{total work costs}.$$

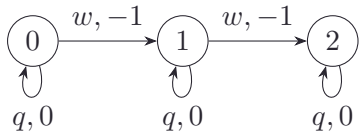
Completion is rewarded, not required: spending effort on an unfinished job can give a negative payoff.

Question: with the same progress, can the best decision change when less time remains?

Model the job by its completed progress

$H = 2$. The state $x \in \{0, 1, 2\}$ is the number of completed units. The same rules apply in both periods:

- At $x < 2$, work w gives reward -1 and moves to $x + 1$.
- Wait q gives reward 0 and leaves progress unchanged.
- At $x = 2$, the job is complete: only wait is feasible.



Edge labels: action, immediate reward.

$$g(0) = g(1) = 0, \quad g(2) = 10.$$

The bonus is paid once at stage 3, after decision 2. There is no third action.

We compute a table for **every** state, not just the initial state $X_1 = 0$.

One decision left: completion may be impossible

Work costs 1 and completes one unit; wait earns 0 and stays. At state 2, only wait is feasible. At stage 3, there are no decisions left, so the remaining payoff is just the bonus:

$$U_3 = g = (0, 0, 10) \qquad \text{in state order } (0, 1, 2).$$

At stage 2, score each action as **reward now + bonus at the resulting state**. The largest score is our candidate value $U_2(x)$.

State	Work score	Wait score	$U_2(x)$	Selected action
0	$-1 + g(1) = -1$	$0 + g(0) = 0$	0	q
1	$-1 + g(2) = 9$	$0 + g(1) = 0$	9	w
2	infeasible	$0 + g(2) = 10$	10	q

From state 0, one work period cannot finish the job: work loses 1, while wait gives 0.

Two decisions left: the same state calls for work

Work costs 1 and advances one unit; wait earns 0 and stays. The terminal payoff is 10 at state 2, and 0 otherwise.

$U_2 = (0, 9, 10)$ gives the values from stage 2 onward: the last reward plus the terminal bonus. At stage 1, use **reward now + value from the next stage onward**. The bonus is already included:

State	Work score	Wait score	$U_1(x)$
0	$-1 + U_2(1) = 8$	$0 + U_2(0) = 0$	8
1	$-1 + U_2(2) = 9$	$0 + U_2(1) = 9$	9
2	infeasible	$0 + U_2(2) = 10$	10

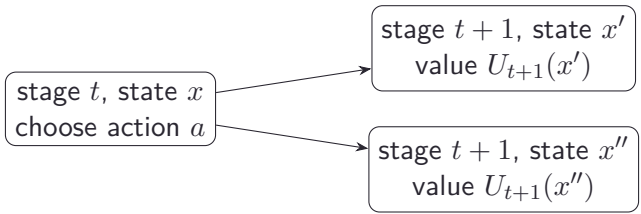
From state 0, work–work gives $-1 - 1 + 10 = 8$; waiting now cannot finish the job and gives 0.

Same state and rules, less time: work at $(t, x) = (1, 0)$, wait at $(2, 0)$.

From the example to a general backward step

$U_{t+1}(x')$ is our candidate **value from the next stage onward**, starting in state x': expected later rewards plus g, excluding R_t.

Even with a certain next state, work at $(t, x) = (1, 0)$ scores $-1 + U_2(1) = 8$.



With **uncertain transitions**, average these values using $\mathcal{P}(x' \mid x, a)$, then add the mean current reward $r(x, a)$.

This is the same one-step extension used in Lecture 4; the deadline now tells us exactly where to start.

One backward step constructs the next table

Suppose the next-stage value table $U_{t+1} : \mathcal{X} \to \mathbb{R}$ is available. For each feasible action, define its score

$$B_t(x, a) := r(x, a) + \sum_{x' \in \mathcal{X}} \mathcal{P}(x' \mid x, a) U_{t+1}(x').$$

The known joint law makes this the expected value of $R_t + U_{t+1}(X_{t+1})$ after selecting a at (t, x).

Construct the candidate value by choosing the largest score:

$$\boxed{U_t(x) := \max_{a \in \mathcal{A}(x)} B_t(x, a), \qquad U_{H+1}(x) := g(x).}$$

At the next stage, there is **one fewer decision** remaining. No discount factor is needed for this finite sum of rewards.

The Bellman equation now links successive stages

Previously: infinite horizon, discounted

$$U(x) = \max_{a \in \mathcal{A}(x)} \left[r(x,a) + \gamma \sum_{x'} \mathcal{P}(x' \mid x, a) U(x') \right].$$

The same U appears on both sides. Discounted Bellman updates converge to U.

Now: finite horizon, undiscounted

$$U_t(x) = \max_{a \in \mathcal{A}(x)} \left[r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x, a) U_{t+1}(x') \right].$$

The next-stage table is already known: start at $U_{H+1} = g$, then compute $U_H, \ldots, U_1$.

Both equations use **reward now + expected future value**. With a deadline, we can work backward from the known endpoint. The next proof shows that $U_t = V_t^*$.

Store a value and a decision at each entry

For every stage t and state x, compute

$$B_t(x, a) = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) U_{t+1}(x').$$

Store two different objects:

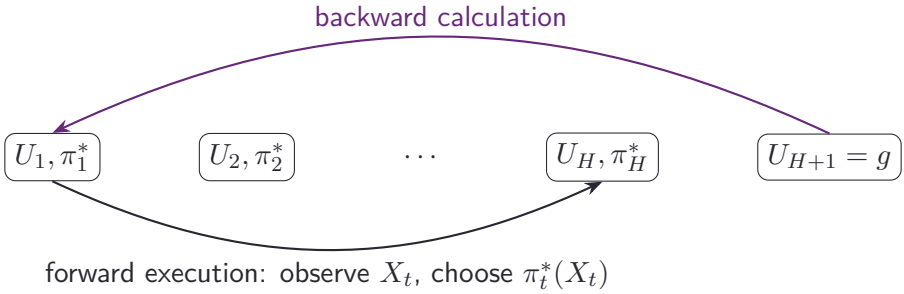
$$U_t(x) = \max_{a \in \mathcal{A}(x)} B_t(x, a) \quad \text{a real number,}$$

$$\pi_t^*(x) \in \operatorname*{argmax}_{a \in \mathcal{A}(x)} B_t(x, a) \quad \text{a feasible action.}$$

Finite nonempty action sets ensure that a maximizing action exists. Fix one choice at every tie to obtain the complete policy sequence $\pi^* = (\pi_1^*, \ldots, \pi_H^*)$.

We will prove that this **one sequence** attains $U_t(x)$ at all stages and states.

Compute backward, act forward



- Calculation uses each complete U_{t+1} table to build U_t.
- Execution uses the entry for the **state actually observed**.
- There are exactly H backward stages; the terminal table is given.

Finite-horizon dynamic programming

1. **Initialize the boundary:** $U_{H+1}(x) \leftarrow g(x)$ for every x.
2. For $t = H, H-1, \ldots, 1$, and every state x:

$$B_t(x,a) \leftarrow r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x,a) U_{t+1}(x') \quad \forall a \in \mathcal{A}(x),$$
$$U_t(x) \leftarrow \max_{a \in \mathcal{A}(x)} B_t(x,a),$$
$$\pi_t^*(x) \leftarrow \text{one maximizing action.}$$

3. **Return** the value tables and the policy sequence $(\pi_1^*, \ldots, \pi_H^*)$.

Use the already completed stage $t+1$ table throughout stage t.

Boundary g $\longrightarrow$ H backward stages $\longrightarrow$ a stage-Markov policy.

Why every history policy is bounded by U_t

Fix any admissible policy ρ, which may use the full history. We prove backward that

$$\mathbb{E}^{\rho}[G_t \mid H_t] \leq U_t(X_t) \quad \text{almost surely.}$$

Boundary: at $H+1$, both sides equal the terminal payoff $g(X_{H+1})$.

Induction step: assume the bound at $t+1$. Then

$$\begin{aligned}
\mathbb{E}^{\rho}[G_t \mid H_t] &\overset{(1)}{=} \mathbb{E}^{\rho}[R_t + G_{t+1} \mid H_t] \\
&\overset{(2)}{=} \mathbb{E}^{\rho}[R_t + \mathbb{E}^{\rho}[G_{t+1} \mid H_{t+1}] \mid H_t] \\
&\overset{(3)}{\leq} \mathbb{E}^{\rho}[R_t + U_{t+1}(X_{t+1}) \mid H_t].
\end{aligned}$$

1. Split the return: $G_t = R_t + G_{t+1}$.
2. Use the tower property. $H_{t+1} = (H_t, A_t, R_t, X_{t+1})$ includes H_t and the received reward R_t.
3. Use the induction hypothesis; conditional expectation preserves inequalities.

Remaining step (next slide): bound this last expectation by $U_t(X_t)$.

Complete the induction step

The previous slide gave $\mathbb{E}^{\rho}[G_t \mid H_t] \le \mathbb{E}^{\rho}[R_t + U_{t+1}(X_{t+1}) \mid H_t]$. **We now bound the right-hand side by $U_t(X_t)$.**

1. Condition on the action. By the model and linearity,

$$\begin{aligned}&\mathbb{E}^{\rho}[R_t + U_{t+1}(X_{t+1}) \mid H_t, A_t] \\ &\quad = r(X_t, A_t) + \sum_{x'} \mathcal{P}(x' \mid X_t, A_t) U_{t+1}(x') = B_t(X_t, A_t).\end{aligned}$$

2. Average over $A_t \sim \rho_t(\cdot \mid H_t)$, using the tower property:

$$\begin{aligned}&\mathbb{E}^{\rho}[R_t + U_{t+1}(X_{t+1}) \mid H_t] \\ &\quad = \sum_{a \in \mathcal{A}(X_t)} \rho_t(a \mid H_t) B_t(X_t, a) \;\le\; \max_{a \in \mathcal{A}(X_t)} B_t(X_t, a) = U_t(X_t).\end{aligned}$$

The weights are nonnegative and sum to 1, so the weighted average of scores cannot exceed their maximum. The last equality defines U_t; reward and next state need not be independent.

Together with the previous slide: $\mathbb{E}^{\rho}[G_t \mid H_t] \le U_t(X_t)$. This completes the induction. Starting at any (t, x), the bound holds for every admissible tail policy ρ; hence $V_t^*(x) \le U_t(x)$.

One fixed stage-Markov policy attains every table

Use the **same stored sequence** $\pi^* = (\pi_1^*, \ldots, \pi_H^*)$ from the algorithm. Prove backward that $V_t^{\pi^*}(x) = U_t(x)$ for every x.

Boundary: $V_{H+1}^{\pi^*}(x) = g(x) = U_{H+1}(x)$.

Step: if the equality holds at $t+1$ for all states, then

$$\begin{aligned} V_t^{\pi^*}(x) &= r(x, \pi_t^*(x)) + \sum_{x'} \mathcal{P}(x' \mid x, \pi_t^*(x)) V_{t+1}^{\pi^*}(x') \\ &= B_t(x, \pi_t^*(x)) = U_t(x). \end{aligned}$$

The first line evaluates the stage-Markov policy π^*; the last uses its stored maximizing action. Together with the upper bound on **all** admissible policies,

$$\boxed{V_t^{\pi^*}(x) = U_t(x) = V_t^*(x) \qquad \text{for every } t, x.}$$

This extends Lecture 4's upper-bound/attainment induction. It ends at the given boundary: no infinite limit or contraction is needed.

The same deadline with uncertain progress

Keep the same two-period job. Now work succeeds with known probability $p = \frac{4}{5}$. Each attempt costs 1, even if it fails.

State	Action	Reward	Next state
$x < 2$	work w	-1	$x+1$ with $\frac{4}{5}$; x with $\frac{1}{5}$
$x < 2$	wait q	0	x, surely
2	wait q only	0	2, surely

$$g(0) = g(1) = 0, \qquad g(2) = 10 \quad \text{at stage } 3 \text{ only.}$$

For $x < 2$, the backward rule becomes

$$\boxed{U_t(x) = \max\Big\{-1 + \tfrac{4}{5}U_{t+1}(x+1) + \tfrac{1}{5}U_{t+1}(x), \quad U_{t+1}(x)\Big\}.}$$

The two entries are the work and wait scores; $U_t(2) = U_{t+1}(2)$.

Last decision: substitute the terminal table

Work costs 1, advances with probability $\frac{4}{5}$, and otherwise stays; wait earns 0 and stays. Only wait is feasible at state 2.

$$U_t(x) = \max\left\{-1 + \tfrac{4}{5}U_{t+1}(x+1) + \tfrac{1}{5}U_{t+1}(x),\ U_{t+1}(x)\right\}, \quad x < 2.$$

Start from $U_3 = g = (0, 0, 10)$. At $t = 2$,

$$\begin{aligned} U_2(0) &= \max\{-1 + \tfrac{4}{5}(0) + \tfrac{1}{5}(0),\ 0\} = 0, \\ U_2(1) &= \max\{-1 + \tfrac{4}{5}(10) + \tfrac{1}{5}(0),\ 0\} = 7, \\ U_2(2) &= 0 + U_3(2) = 10. \end{aligned}$$

$$\boxed{U_2 = (0, 7, 10), \qquad (\pi_2^*(0), \pi_2^*(1), \pi_2^*(2)) = (q, w, q).}$$

At state 1, work gives an $\frac{4}{5}$ chance of the payoff 10, at a sure cost of 1.

First decision at state 0: average future decisions

Work costs 1, advances with probability $\frac{4}{5}$, and otherwise stays; wait earns 0 and stays. The solved stage-2 table is $U_2 = (0, 7, 10)$.

$$U_1(0) = \max\left\{-1 + \tfrac{4}{5}U_2(1) + \tfrac{1}{5}U_2(0),\ U_2(0)\right\}.$$

$$\begin{aligned}\text{work score} &= -1 + \tfrac{4}{5}(7) + \tfrac{1}{5}(0)\\ &= \tfrac{23}{5} = 4.6,\\ \text{wait score} &= 0.\end{aligned}$$

If work succeeds, state 1 remains: work at stage 2, worth 7.
If work fails, state 0 remains: wait at stage 2, worth 0.

$$\boxed{U_1(0) = 4.6, \qquad \pi_1^*(0) = w.}$$

The next-stage value table already includes the best action **after observing** the outcome.

Complete the tables and the policy

With $p = \frac{4}{5}$, work costs 1 and advances one unit on success; wait earns 0 and stays. Only wait is feasible at 2. Using $U_2 = (0, 7, 10)$, the remaining first-stage entries are

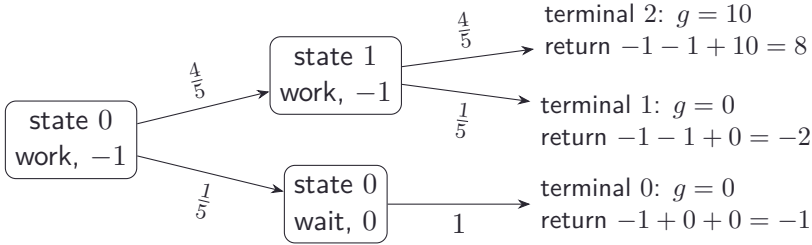
$$\begin{aligned} U_1(1) &= \max\{-1 + \tfrac{4}{5}U_2(2) + \tfrac{1}{5}U_2(1),\ U_2(1)\} \\ &= \max\{-1 + 8 + 1.4,\ 7\} = 8.4, \\ U_1(2) &= 0 + U_2(2) = 10. \end{aligned}$$

	Value $U_t = V_t^*$			Selected action π_t^*		
Stage	0	1	2	0	1	2
3	0	0	10		no decision	
2	0	7	10	q	w	q
1	4.6	8.4	10	w	w	q

State 0 demonstrates why the optimal policy needs the stage index.

Check the actual return of the stage-Markov policy

Start from $X_1 = 0$, with $p = \frac{4}{5}$. Work first; at stage 2, work if state 1 and wait if state 0.



$$\mathbb{E}[G_1] = \left(\tfrac{4}{5}\right)^2 (8) + \tfrac{4}{5}\tfrac{1}{5}(-2) + \tfrac{1}{5}(-1) = \tfrac{23}{5} = 4.6 = U_1(0).$$

Each term is a path probability times its total return, including the terminal bonus **once**.

What must a state remember?

A state is useful for DP when it retains the information needed for future decisions.

For two observed histories with the same (t, x), check that:

- the same actions are feasible;
- under each action, the conditional law of (R_t, X_{t+1}) is the same;
- the state at the deadline determines the terminal payoff.

In the progress model

$x \in \{0, 1, 2\}$ records completed progress. The stage t records how many decisions remain. If remaining resources affect feasibility or dynamics, they must also be recorded in the state.

The state can summarize the history even when admissible policies are allowed to use the full history. In our homogeneous model, the feasible actions and physical joint law also stay the same across stages.

The same physical state, a different stage

Model: $H = 2$, $x \in \{0, 1, 2\}$. For $x < 2$, work earns -1 and advances by one with probability 0.8, otherwise stays; wait earns 0 and stays. At $x = 2$, only wait is feasible.

$$g(0) = g(1) = 0, \qquad g(2) = 10 \quad \text{at stage } 3 \text{ only.}$$

At progress $x = 0$, backward DP gives:

Current stage	Decisions left	Optimal action
$t = 1$	2	work
$t = 2$	1	wait

One remaining decision cannot reach progress 2 from progress 0.

Equivalently, augment the state to $z = (t, x)$. A stationary rule $\mu(z) = \pi_t(x)$ uses the clock through z: equal progress at different stages gives different augmented states.

What does a complete backward DP calculation cost?

Let $n = |\mathcal{X}|$, with at most m feasible actions at each (t, x). DP computes

$$U_t(x) = \max_{a \in \mathcal{A}(x)} \left\{ r(x, a) + \sum_{x' \in \mathcal{X}} \mathcal{P}(x' \mid x, a) U_{t+1}(x') \right\}.$$

Work to perform	Worst-case count
Action scores over all stages and states	$O(Hnm)$ backups
Successor average in one dense backup	$O(n)$ arithmetic
All dense backups	$\boxed{O(Hn^2m)}$ arithmetic

All values and actions need $O(Hn)$ storage; consecutive value layers need $O(n)$.

For k coordinates with s values each, $n = s^k$: adding coordinates can be costly.

Which difficulty are we trying to address?

What changes?	Main response	Course connection
The task has a deadline	Use stage-dependent values and backward DP	Finite-horizon DP
Solving every state is too expensive	Concentrate planning on the current decision	Online planning
A separate value for every state is impractical	Share structure through function approximation	Linear approximation; DQN
Rewards or transitions are unknown	Learn a model, values, or a policy from samples	MC, TD, Q-learning; exploration

Our next step: keep the known model, but focus computation on the action needed now.

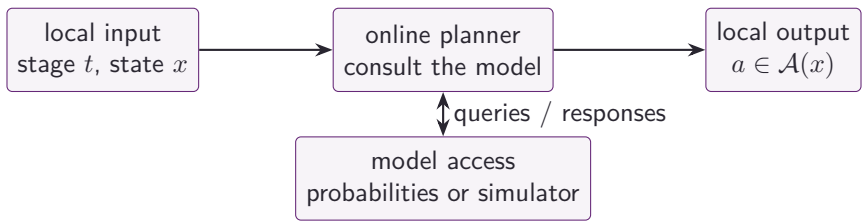
The other rows will help locate later methods; these difficulties can occur together.

Online Planning
From the State Observed Now



Compute an action on demand; implement a policy

Complete backward DP prepares actions for **every stage and state**. An online planner instead receives the observed (t, x) and returns **one feasible action per call**.



At each subsequent decision, call the planner again at the new stage and state. This implements a policy **without listing its action at every possible input**.

The objective is still the expected return over *all remaining decisions*, not just the current reward.

Two ways to use a known model

Use the model to calculate an expectation or estimate it from simulated outcomes.

Explicit probabilities

Use the mean reward and all next-state probabilities:

$$r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) C(x').$$

$C(x')$ is a supplied value from the next stage onward.

Simulator / generator

From an imagined (x, a), draw

$$x' \sim \mathcal{P}(\cdot \mid x, a).$$

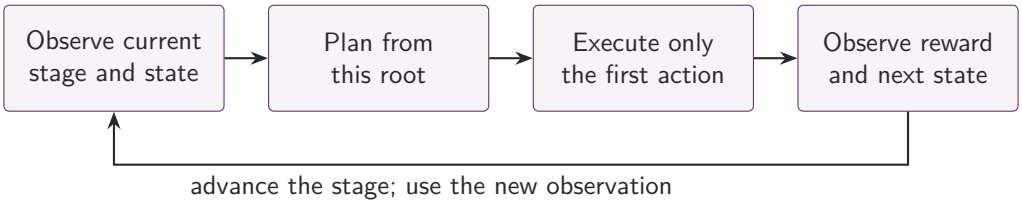
Use the known mean reward $r(x, a)$. Average returns from repeated simulated paths.

First: build a search tree from the current state and compute expectations from probabilities.

Then: estimate returns from simulated episodes and introduce MCTS.

Observe, plan, execute one action, and repeat

A search examines alternative futures. The real system follows only the action we execute and the outcome that occurs.

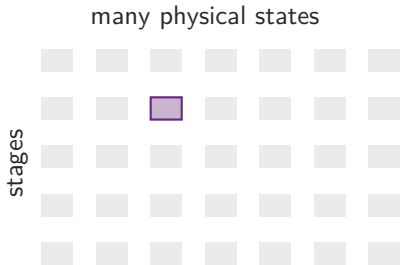


The deadline H stays fixed; the stage advances and fewer decisions remain.

Later decisions can react to the observed outcome; we do not commit to a fixed action sequence.

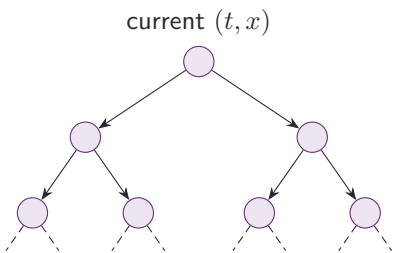
Focus the calculation on futures reachable from here

A complete stage–state table



Prepare decisions even for states we may never reach.

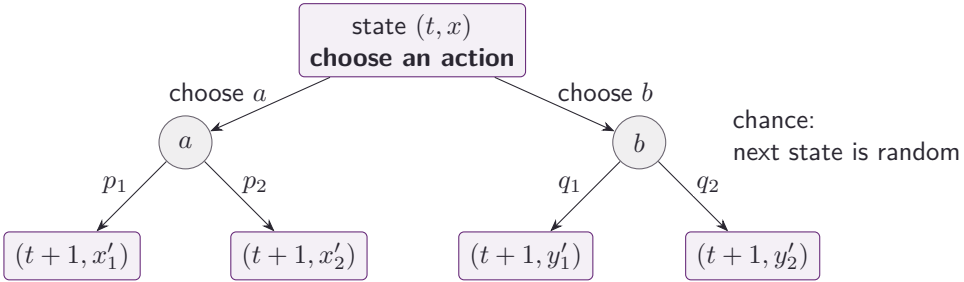
A tree rooted at the observed (t, x)



Only possible futures from this root. Actions and outcomes both branch.

This may avoid much of the full table. It is **not automatically cheaper**: a large or deep tree can repeat the same (stage, state) many times.

Two kinds of branching: our choice and random outcomes



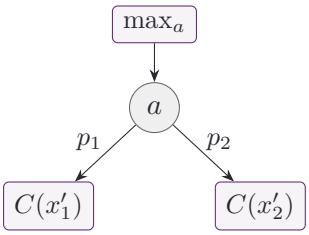
Decision nodes: maximize over feasible actions.

Chance nodes: average with known probabilities $p_i = \mathcal{P}(x_i' \mid x, a)$ or $q_i = \mathcal{P}(y_i' \mid x, b)$.

At the next state, choose a new action after observing which state occurred.

Search uses the Bellman calculation we already know

At each next-state node, suppose a value $C(x')$ from that stage onward is available.



1. Average after a fixed action:

$$\text{score}(a) = r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x, a) C(x').$$

2. Choose the largest score:

$$\text{value at } (t,x) = \max_{a \in \mathcal{A}(x)} \text{score}(a).$$

The reward $r(x,a)$ is the expected current reward; no independence between reward and next state is needed.

With $C = U_{t+1}$, this is exactly the backward-DP backup. A tree applies it from its leaves back to its root.

Reuse our job model to choose an action at the root

Job: earn a bonus if two units are complete after two decisions. State $x \in \{0, 1, 2\}$ counts completed units; currently $(t, x) = (1, 0)$.

Action	Current reward	Next state
Work w, if $x < 2$	-1, success or failure	$x + 1$ with probability 0.8; else x
Wait q	0	x with probability 1

At $x = 2$, only wait is feasible. The only completion bonus is at stage 3:

$$g(0) = g(1) = 0, \qquad g(2) = 10.$$

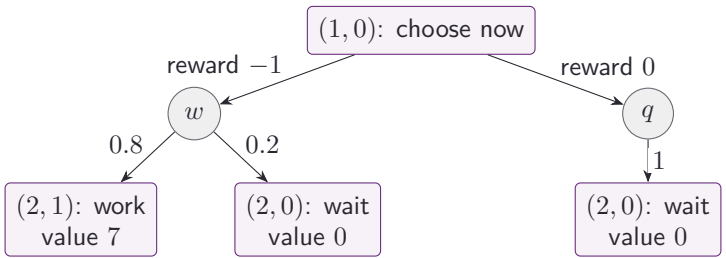
Our earlier one-decision calculation gave

$$\boxed{U_2(0) = 0, \quad U_2(1) = 7, \quad U_2(2) = 10.}$$

For example, at $x = 1$, work has score $-1 + 0.8(10) + 0.2(0) = 7$, versus 0 for wait.

Back up the possible next decisions to the current root

At $(1, 0)$, work costs 1 and advances with probability 0.8; wait stays for free. The stage-2 values are $U_2(0) = 0$, $U_2(1) = 7$.



$$\underbrace{-1 + 0.8(7) + 0.2(0)}_{\text{work score}} = \boxed{4.6} \quad > \quad \underbrace{0 + U_2(0)}_{\text{wait score}} = 0.$$

Choose work now. Afterward, the observed next state determines the next action.

A search plan includes reactions, not just an action sequence

Same job: $H = 2$, start $x = 0$; work costs 1, succeeds with probability 0.8; wait is free. Completing two units earns 10 at stage 3.

Fixed sequence: work, then work

Pay both costs. Receive the bonus only after two successes:

$$-2 + 10(0.8)^2 = \boxed{4.4}.$$

Feedback: work, then react

Work again after success; wait after failure:

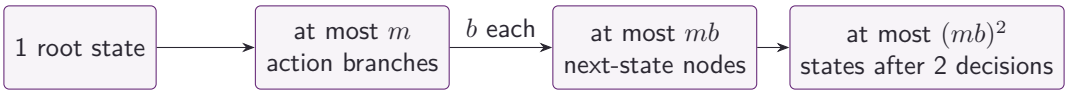
$$-1 - 0.8 + 10(0.8)^2 = \boxed{4.6}.$$

After a first failure, completion is impossible. Waiting saves the unnecessary second cost of 1 on a branch with probability 0.2.

The tree maximizes separately at each possible next state, *before* averaging their values at the current root.

A rooted tree can still be too large

With at most m actions and b possible next states per action:



A naive tree through d decisions has at most

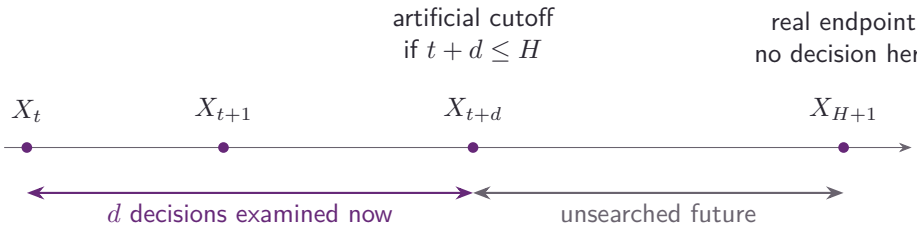
$$1 + mb + \cdots + (mb)^d \quad \text{state nodes: } O((mb)^d) \text{ for } mb > 1.$$

- Cache repeated $(\text{stage}, \text{state})$ values to avoid duplicate work.
- Limit depth, or sample paths instead of expanding every branch.

Our tiny job illustrates the calculation; meaningful savings need a larger state space.

A search cutoff is different from the real deadline

At stage t, search d decisions ahead, where $1 \leq d \leq H - t + 1$.



- One decision means choose an action, receive its reward, and reach the next state. The chance node is not another decision.
- If the search reaches $H + 1$, use the true terminal payoff g.
- If it stops earlier, the leaf still has rewards ahead. We need an estimate of that remaining return.

A cutoff value stands in for the unsearched future

At an early cutoff stage $t + d$, supply a function $W_{t+d}(x')$ estimating the remaining return from leaf state x'.

	What does the leaf number mean?
True endpoint $g(x')$	Part of the task. Paid once at the actual stage $H + 1$.
Early cutoff $W_{t+d}(x')$	Part of the planning method. Estimates stage rewards from $t + d$ through H, plus the eventual g. It is not a reward received there.

$$\text{one-step score} = r(x, a) + \sum_{x'} \mathcal{P}(x' \mid x, a) W_{t+1}(x').$$

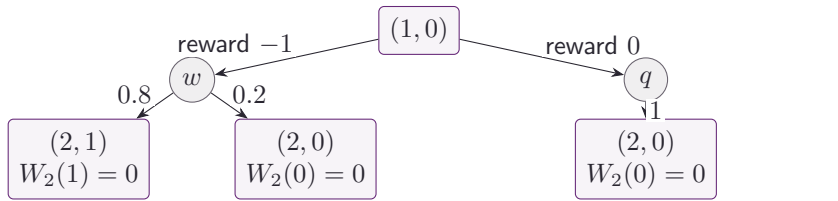
Even when $g \equiv 0$, a cutoff value may be nonzero because stage rewards remain.

W may be supplied by a heuristic, a stored value table, or the value of a feasible base policy. No learning method is required to define it.

Cut the same job tree after one decision

Recall: at $(1, 0)$, work earns -1 and reaches 1 with probability 0.8; wait earns 0 and stays. The 10-unit completion bonus is at stage 3.

Depth $d = 1$ ends at stage 2. Set its cutoff values to zero: $W_2 \equiv 0$.



$$\text{work: } -1 + 0.8(0) + 0.2(0) = -1 \quad < \quad \text{wait: } 0 + 0 = 0.$$

Choose wait. Zero leaf values erase the benefit of beginning the job.

Replanning cannot recover the lost first decision

After executing wait at stage 1, $X_2 = 0$. Only one real decision remains.

Same model: work costs 1, adds one unit with probability 0.8, and otherwise stays. Wait is free. Only $x = 2$ earns the terminal bonus 10.

Now one-step search reaches the real endpoint, so it uses g, not W:

$$\begin{aligned} \text{work:} \quad & -1 + 0.8g(1) + 0.2g(0) = -1, \\ \text{wait:} \quad & 0 + g(0) = 0. \end{aligned}$$

It waits again. The actual return under this replanning rule is

$$\boxed{0 + 0 + g(0) = 0},$$

whereas the optimal expected return from $(1, 0)$ is 4.6.

A new search uses new observations; it cannot undo an earlier action that left too little time.

Two ways to account for the unsearched future

Same root $(1, 0)$, same job. The problem was not the current reward or transition model; it was the treatment of what happens after the cutoff.

Search deeper

Use $d = 2$. Reach the actual endpoint g and back up the second decisions.

$$\text{work:} \quad -1 + 0.8(7) + 0.2(0) = 4.6.$$

This repeats the exact depth-2 calculation we just saw.

Use informative leaf values

Keep $d = 1$, but use $W_2 = U_2$:

$$W_2(0) = 0, \quad W_2(1) = 7.$$

The same root scores result without searching the last decision now.

For this job both methods choose work. In general, a deeper search with arbitrary cutoff values need not improve the real policy.

Use the same calculation for any search depth

In our job example, we used g after two decisions, or W_2 after one. Now repeat the same steps for any depth $d \geq 1$.

From the current (t, x), search to stage $t + d \leq H + 1$. Let $Z_s(y)$ be the value this search computes for state y at stage s.

1. **Start at the last states in the search.**
   - At the real deadline $H + 1$: set $Z_{H+1}(y) = g(y)$.
   - If the search stops earlier: set $Z_{t+d}(y) = W_{t+d}(y)$.
2. **Work backward**, for $s = t + d - 1, \ldots, t$:

$$Z_s(y) = \max_{a \in \mathcal{A}(y)} \left[r(y, a) + \sum_{y'} \mathcal{P}(y' \mid y, a) Z_{s+1}(y') \right].$$

3. **Choose an action with the largest score**, at the current (t, x).

Before the stopping stage, include every feasible action and every next state that can occur. Approximate W values can change the chosen action.

What changes, and what stays the same?

Same Bellman logic	Maximize over actions; average over random outcomes.
Different computation	Start from the current state and compute when an action is needed. Execute one action, observe, and replan.
Finite search budget	Limit depth and supply values for the unsearched future; a rooted tree can still have too many branches.

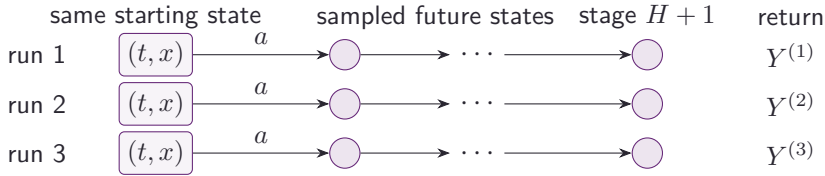
So far: enumerate the possible outcomes

We used known probabilities to calculate expectations exactly. The difficulty was computation, not missing model information.

Next question: if the known model can generate random outcomes, can we estimate returns from a manageable number of simulated paths instead of expanding every branch?

Sample paths and average their returns

Sample paths instead of expanding every branch. Restart each run at the current (t, x), using the same first action a and a fixed stage-Markov policy afterward.



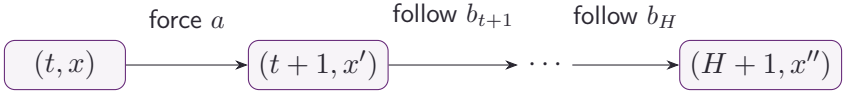
At each step, draw the next state from $\mathcal{P}$ and add the known mean reward r. At $H+1$, add g once. Each $Y^{(k)}$ is one simulation's total return.

Repeat independently and average the returns. This estimates the expected return of action a followed by the fixed policy. Choose how many simulations to run; a finite average is an approximation.

The real system stays in its current state while we simulate.

After the first action, follow a fixed policy

Compare action a at (t, x), then follow a fixed stage-Markov rule b. Its action probabilities are $b_s(\cdot \mid x_s)$.



Use $c_s = r(x_s, a_s)$. Run K independent simulations; $Y^{(k)}$ is run k's return:

$$Y = \sum_{s=t}^{H} c_s + g(x_{H+1}), \qquad \widehat{Q}_t^b(x, a) = \frac{1}{K} \sum_{k=1}^{K} Y^{(k)}.$$

$Q_t^b(x, a)$ denotes the expected return when we start at (t, x), force a, then follow b. The terminal payoff appears **once**.

For independent bounded runs with fixed b, the law of large numbers gives $\widehat{Q}_t^b(x, a) \xrightarrow[K \to \infty]{} Q_t^b(x, a)$, not necessarily the optimal action value.

What does a rollout average estimate in our job model?

Model: two decisions; initial progress 0. Below progress 2, work costs 1 and advances one unit with probability 0.8; wait costs 0 and stays. At progress 2, only wait is feasible. Stage-3 payoff: $g = (0, 0, 10)$.

Force work first; then use b: **work whenever the job is unfinished**.

Two simulated work outcomes	Probability	Final progress	Return
success, success	$0.8^2 = 0.64$	2	$-1 - 1 + 10 = 8$
all other outcomes	$1 - 0.64 = 0.36$	0 or 1	$-1 - 1 + 0 = -2$

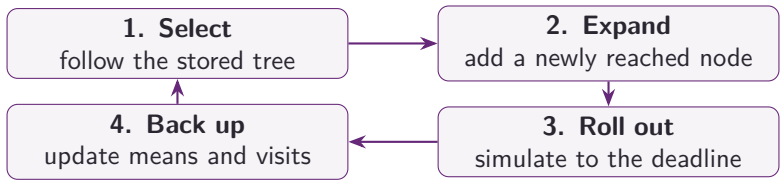
Thus repeated rollouts estimate

$$Q_1^b(0, w) = 0.64(8) + 0.36(-2) = \boxed{4.4}.$$

The optimal work score was 4.6: after an initial failure, it *waits* rather than paying for futile work. More simulations of b do not remove that policy difference.

MCTS grows a useful part of the tree

Monte Carlo tree search (MCTS) stores simulated action returns inside a growing tree to guide later simulations.



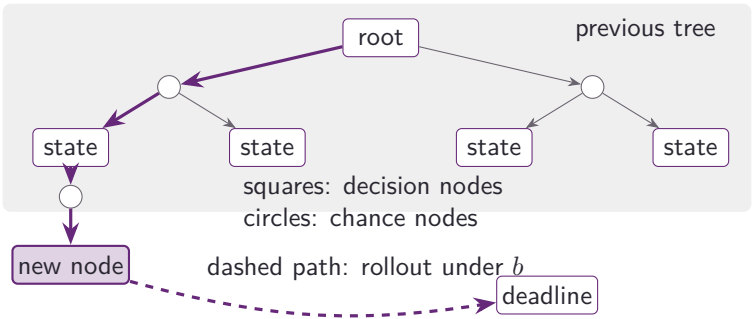
Each node represents a simulated history ending at (s, x). The same state can appear on different branches; only part of the tree is stored.

Choose actions, but **draw random outcomes from** $\mathcal{P}$. The planner cannot choose a favorable outcome.

Adapted from CMU Deep RL planning slides; stochastic-MDP diagram redrawn.

One growing tree, with a simulated path through it

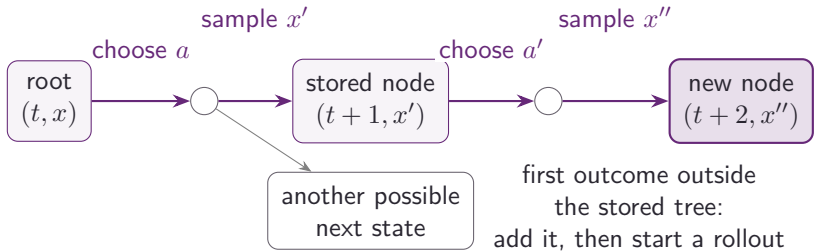
Shading shows the tree before this simulation. The highlighted path adds the purple frontier node to it, then continues with an unstored rollout.



The next simulation starts at the same root, with the larger stored tree and updated statistics. Tree/frontier/rollout illustration adapted from CMU Deep RL planning slides.

Selection chooses actions; expansion records a new outcome

At stored nodes, try untried actions; otherwise balance return estimates and visit counts.

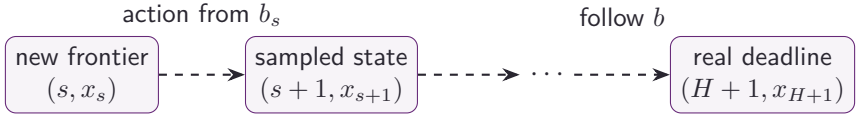


Stored information: for each tree node n and tried action a, keep a visit count $N(n, a)$ and an empirical mean return $\overline{Q}(n, a)$.

This is the MCTS framework; its precise action-selection rule is a design choice. Exploration bonuses can formalize the tradeoff; their analysis comes later.

Beyond the stored frontier, finish a simulated episode

After expansion, draw actions from a simple rollout policy $b_s(\cdot \mid x_s)$, such as uniform random feasible actions. It need not be optimal.

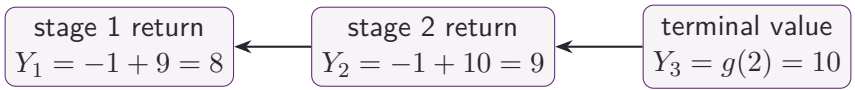


- Collect simulated rewards through stage H, then add $g(x_{H+1})$ once.
- At an earlier cutoff k, replace the remaining rollout by $W_k(x_k)$. Setting $W_k = V_k^b$ preserves the fixed-b expected return.
- The rollout need not add all its states to the stored search tree.

All these are imagined decisions. The real state stays fixed while we run more simulations from the same root.

Back up the remaining return at each node

One successful job simulation has $x_1 = 0 \to x_2 = 1 \to x_3 = 2$, work rewards $-1, -1$, and terminal payoff 10.



In general, set $Y_{H+1} = g(x_{H+1})$, then $Y_s = c_s + Y_{s+1}$. For a visited tree node n at stage s and its selected action a, update

$$N(n,a) \leftarrow N(n,a) + 1, \qquad \overline{Q}(n,a) \leftarrow \overline{Q}(n,a) + \frac{Y_s - \overline{Q}(n,a)}{N(n,a)}.$$

Use the **new count** in the denominator. Initialize the first mean with its first return.

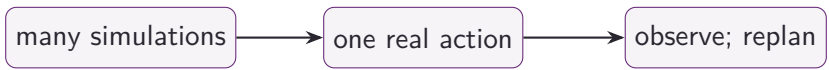
This updates averages, not neural gradients. Adaptive tree decisions change the policies used in later simulations: these are not independent samples from one fixed policy.

After the simulation budget, execute one real action

Illustrative root statistics after 100 simulations:

Root action	Visits	Mean simulated return
work	80	4.5
wait	20	-0.1

Choose the highest root mean, or use the common alternative of choosing the most-visited action. Both select work here.



At the observed $(t+1, x')$, retain the matching subtree or build a new one.

A finite budget does not certify optimality: rare events and poor rollouts can mislead.

The same planning problem, different uses of computation

Method	Where computation goes	How futures are evaluated
Complete backward DP	Every stage and state	Exact successor averages
Online lookahead	A tree from the current state	Exact averages; g or a cutoff value at leaves
Simulation / MCTS	Sampled futures from the current state	Sample returns; selective tree growth

All three compare **current reward plus future return**. They differ in which futures they compute and how accurately they evaluate them.

Simulation-based planning needs a simulator, not prior RL

Our requirements:

- a generator queryable at imagined states;
- finite, nonempty feasible action sets;
- a finite deadline and bounded rewards/payoff;
- an action-selection rule, a rollout policy and a simulation budget.

The model can be known. **No MC, TD or Q-learning algorithm is required.** We have only used simulated trajectories and arithmetic averages.

"Monte Carlo" means **estimating expectations by random samples**. Here, samples support decision-time search using a trusted model.

Later methods update values or policies from experience without requiring explicit known r and $\mathcal{P}$. That experience can come from a sampling oracle or from trajectory interaction; their data-access constraints differ.

From Planning to Learning



Function approximation: share information across states

A table stores a separate number for each stage–state pair.

$$V_t(x) \quad \longrightarrow \quad V_\theta(t, x).$$

V_θ is a chosen family of functions; θ contains shared parameters.

- Features or a neural network share parameters across states.
- Updating θ can change predictions at states not directly observed. A useful representation makes this generalization possible.

We may approximate V, Q, or π. MC, TD and Q-learning can use tables or function approximation.

Search chooses where to compute; approximation chooses a representation. A table or an approximator can provide leaf values.

Reinforcement learning: obtain knowledge from experience

Our DP calculations used known r and $\mathcal{P}$:

$$r(x,a) + \sum_{x'} \mathcal{P}(x' \mid x,a) V_{t+1}(x').$$

If they are unknown, we can use sampled experience (X_t, A_t, R_t, X_{t+1}).

Learn a model, then plan
Estimate rewards and transitions from data, and use them in planning.
This is a model-based approach to RL.

Learn values or a policy directly
Use observed experience without explicitly fitting a transition model.
Next: MC, TD and Q-learning.

Simulation did not make the model unknown. The search we just studied used a trusted model. Here, we must infer useful predictions from experience.

That experience may come from a sampling oracle or trajectory interaction.

Two ways to obtain samples of an unknown model

Generative sampling oracle

- Choose a state x and available action a.
- Receive a fresh reward and next-state sample (R, X') from the true model at that input.
- Repeat the query or choose another input without first reaching that state.

Which queries, and how many, give accurate estimates?

Trajectory interaction

- Start from the task's initial state or initial-state distribution.
- At the current X_t, choose A_t; observe R_t and X_{t+1}.
- Follow actual transitions; reset only as allowed by the task. Reaching a state may require several earlier actions.

How do we learn good decisions while collecting informative experience?

As with our planning generator, an oracle removes the need to **reach a queried state**. Here the probabilities are unknown: samples help estimate them, and query counts still matter.

Exploration under trajectory interaction

Suppose the task's work success probability p is unknown.

- A sampling oracle lets us request working outcomes at a chosen state.
- During trajectory interaction, working changes progress and reveals success or failure. Waiting provides no working outcomes.

For control, an action affects both **return** and **the data used for future decisions**.

Bandits study the reward–information tradeoff without controlled state transitions. MDPs also require reaching states where useful information can be obtained.

Why finite-horizon DP will return

Here, an episode is one complete H-step run. Later methods can use data from earlier episodes to update stage values backward, then act forward through the next episode.

These choices can be combined

Example	Choices being combined
Tabular Q-learning	Learning from experience + a value table
Deep Q-learning (DQN)	Learning from experience + a neural action-value function
LSVI-UCB (later)	Backward regression + linear features + data + exploration
Search with learned values	Decision-time planning + learned leaf estimates

LSVI-UCB: least-squares value iteration with upper confidence bounds.

How to obtain a policy: derive actions from value estimates, or optimize policy parameters directly (policy gradients). Actor–critic methods also learn a value estimate.

Online planning computes the current decision, possibly using simulated episodes. **Online learning** gathers information while interacting with the environment.

What to take away

1. With unchanged reward and transition rules, a deadline can change values and optimal decisions. Keep the stage in values and decision rules.
2. Backward DP solves shorter tail problems first. Its finite proof reuses our earlier upper-bound and attainment arguments.
3. Online planning concentrates computation on the current decision. Search depth and values assigned to the unsearched future determine what it can see.
4. A known model can generate simulated episodes. Rollout averages evaluate the policy used after the first action; MCTS adapts decisions in a growing tree.
5. Representation, model knowledge and data collection raise different questions. Later methods address these questions and can be combined.

Next: when the model is unknown, how can sampled experience replace exact model-based calculations?